

Title: SoundVault

https://github.com/kalvarioz/CST_336_final

<Disclaimer> - Piracy is a crime, this program does not in any way encourage, allow, or distribute pirated content, all sound sources are sourced and owned physically and owned privately. This is for educational purposes only, and songs will be removed from all storages and will no longer function after grading to ensure security.

Project Description:

SoundVault is a web-based music player inspired by offline, local music players such as Foobar2000, Strawberry, and AIMP, but with a mixture of streaming service capabilities. It allows authenticated users to browse a curated music library, stream tracks securely via a server-side proxy, create and manage personal playlists, and discover music through album art and metadata pulled from the MusicBrainz API, the music must be by one user, who hosts the server and the music, which will allow others to play music and create playlists.. A local Express API serves as the music catalog and streaming backend, while all audio files are stored in cloud storage and are never directly exposed to the browser.

Functionality:

- Users can create accounts, and log in to their accounts
- Users can create playlists, and add songs from their library to the playlists,
- Users can modify their profile and add a display name.
- Users can delete their own accounts.

API Fetch calls:

There was only one API that was used as another was not fully finished.

MusicBrainz API: obtained information from songs in the database, enriched the data with missing information and matched the song title/album with the corresponding ID in MusicBrainz.

2nd API: were the login itself (to authenticate levels of access to the database), scripts to import, enrich and obtain songs. The code is present in the program, but is not an NPM package.

How it works:

How a Song Gets Played

When a user clicks a song in the library, the following sequence occurs:

- 1. Browser sends GET /api/songs/stream/:id:** The audio element's src attribute is set to this URL. The browser includes the session cookie automatically.
- 2. requireLogin middleware checks the session:** If the user isn't logged in, the server returns 401 Unauthorized. The browser never reaches R2.
- 3. Server queries MySQL for the cloud_key:** The song_id maps to a cloud_key like "music/audioslave---cochise.flac". This key is never sent to the browser.
- 4. Server sends GetObjectCommand to R2:** Using server-side credentials (stored in environment variables), Express fetches the audio file from Cloudflare R2. The Range header is passed through for seek support.
- 5. R2 response is piped directly to the browser:** The audio stream flows through Express without being buffered in memory. Content-Type, Content-Length, and Content-Range headers are forwarded. Content-Disposition is set to "inline" to prevent downloads.
- 6. Browser's <audio> element plays the stream:** The user hears the song. The HTML5 audio controls show progress, volume, and seeking. The download button is hidden via controlsList="nodownload" and CSS.

This proxy pattern means the browser only ever sees URLs like /api/songs/stream/5. It never sees the R2 endpoint URL, the bucket name, or any AWS credentials. Even if a user opens the Network tab in DevTools, they see the Express URL, not the Cloudflare URL.

Endpoints:

For our connection with the API the following design was implemented through two files songsAPI, and authAPI:

Method	Endpoint	Description
POST	/api/auth/signup	Create account (INSERT into users, bcrypt hash)
POST	/api/auth/login	Authenticate (SELECT + bcrypt compare, set session)
POST	/api/auth/logout	Destroy session, redirect to login
GET	/api/songs	List songs with optional search + genre filter (FULLTEXT)
GET	/api/songs/stream/:id	Streaming proxy: session check, MySQL lookup, R2 pipe
GET	/api/songs/featured	Random enriched songs for the home page marquee
GET	/api/genres	Dynamic genre list (SELECT DISTINCT from songs)
POST	/api/songs/enrich/:id	Auto-enrich one song via MusicBrainz (UPDATE)
POST	/api/songs/enrich-all	Bulk enrich all un-enriched songs
GET	/api/user/profile	Fetch logged-in user's profile
PUT	/api/user/profile	Update 3 fields (display_name, email, favorite_genre)
GET	/api/musicbrainz/search	Proxy search to external MusicBrainz API
GET	/api/musicbrainz/release/:id	Proxy lookup to external MusicBrainz API
GET	/api/admin/users	List all users (admin only)
PUT	/api/admin/users/:id	Edit user (UPDATE 4 fields, admin only)
DELETE	/api/admin/users/:id	Delete user (admin only)
GET	/api/admin/songs	List all songs with cloud_key (admin only)
PUT	/api/admin/songs/:id	Edit song metadata (admin only)
POST	/api/admin/songs	Add song record (INSERT, admin only)
GET	/api/admin/r2/files	List R2 bucket contents (admin only)
DELETE	/api/admin/r2/files	Delete R2 object (admin only)

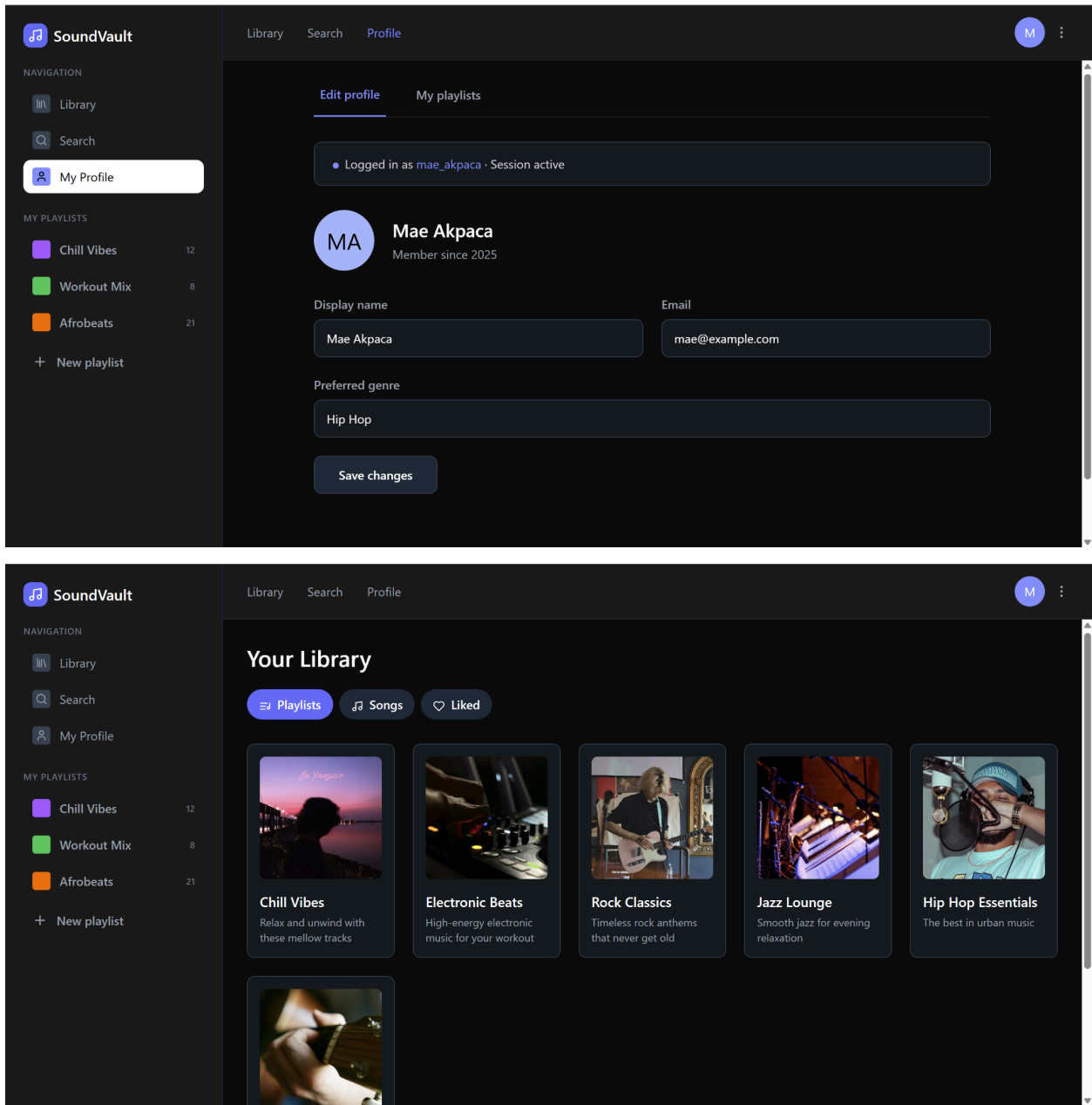
Test accounts:

Name	Password
brandon123	s3cr3t123
demo_user	s3cr3t
test_user	s3cr3t

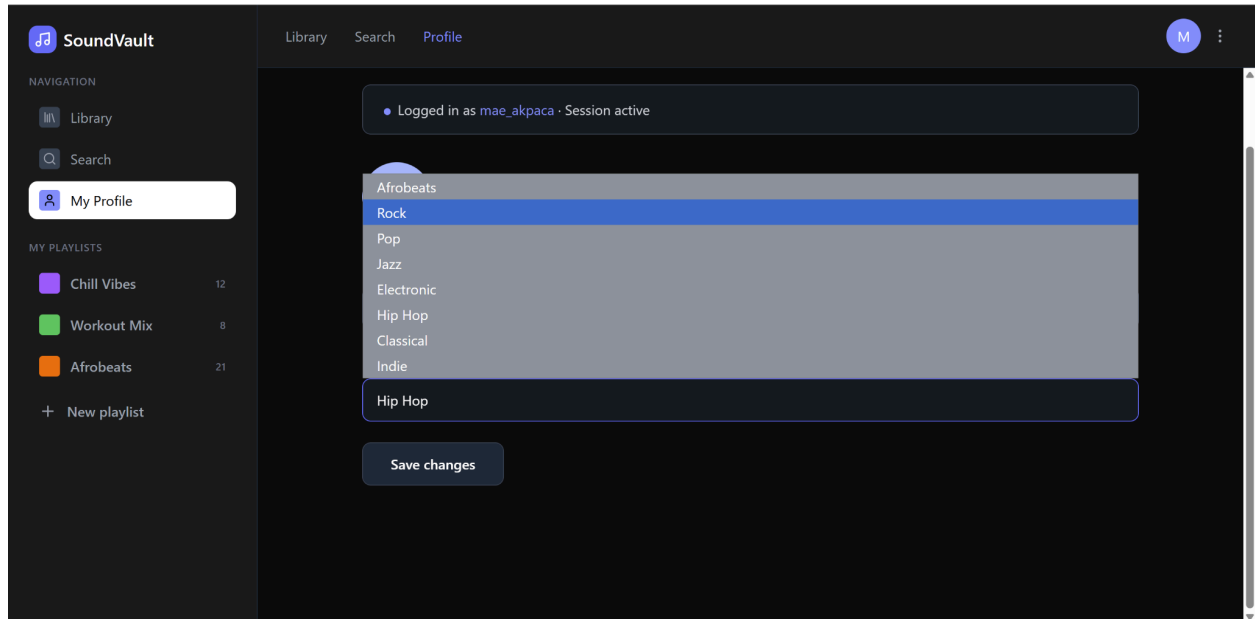
Target Audience:

Our target audience is music enthusiasts or users who want a personal, self-hosted-style listening experience on the web, people who value organized libraries, playlist creation, and secure streaming without the complexity of managing desktop software. This includes students, hobbyist collectors, and anyone who prefers a clean, lightweight player over commercial streaming platforms.

Proposed Mockup:

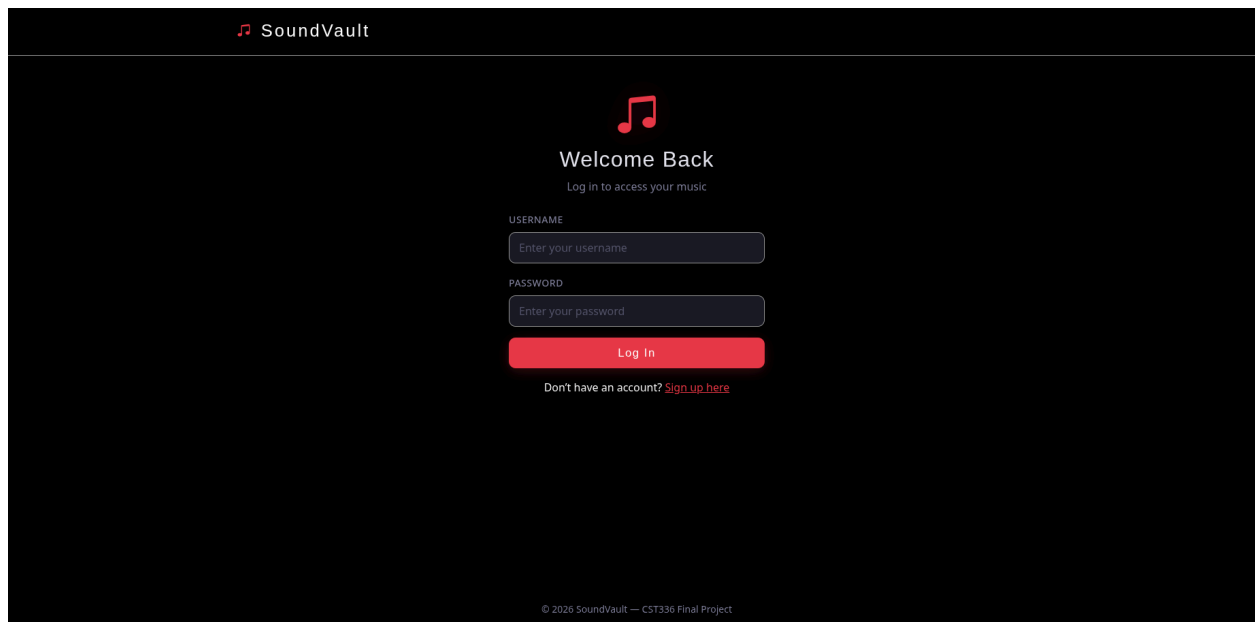


Brandon Calvario, Matthew Barrett, Maeva Akpaca, Dalia Cabrera Hurtado



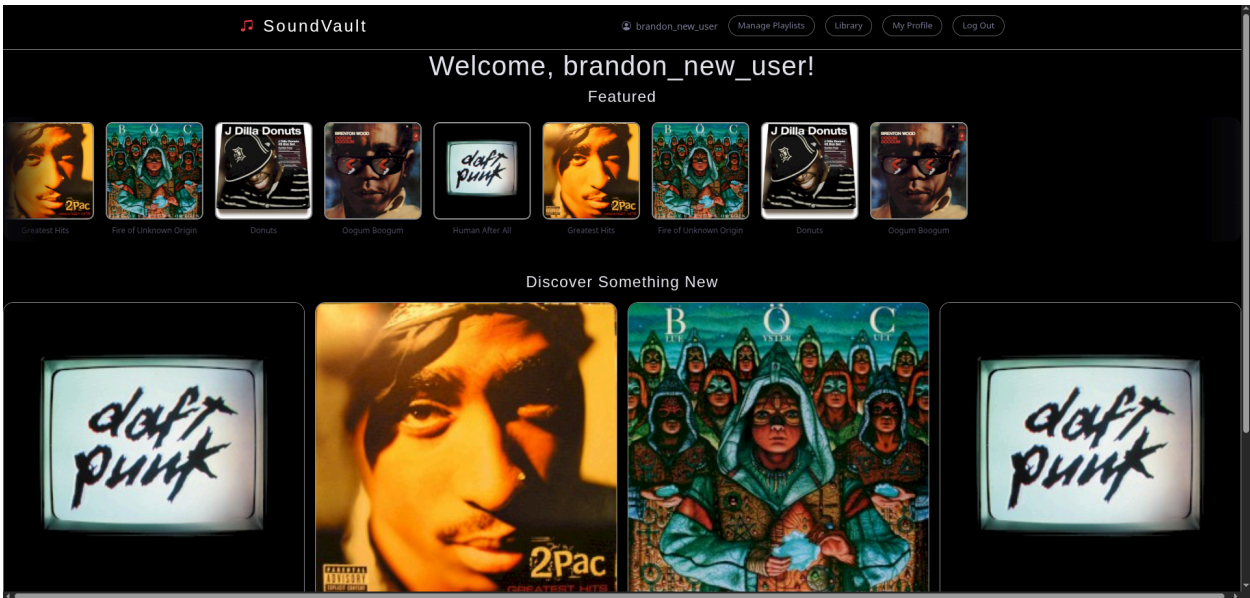
Final Design:

Login page:

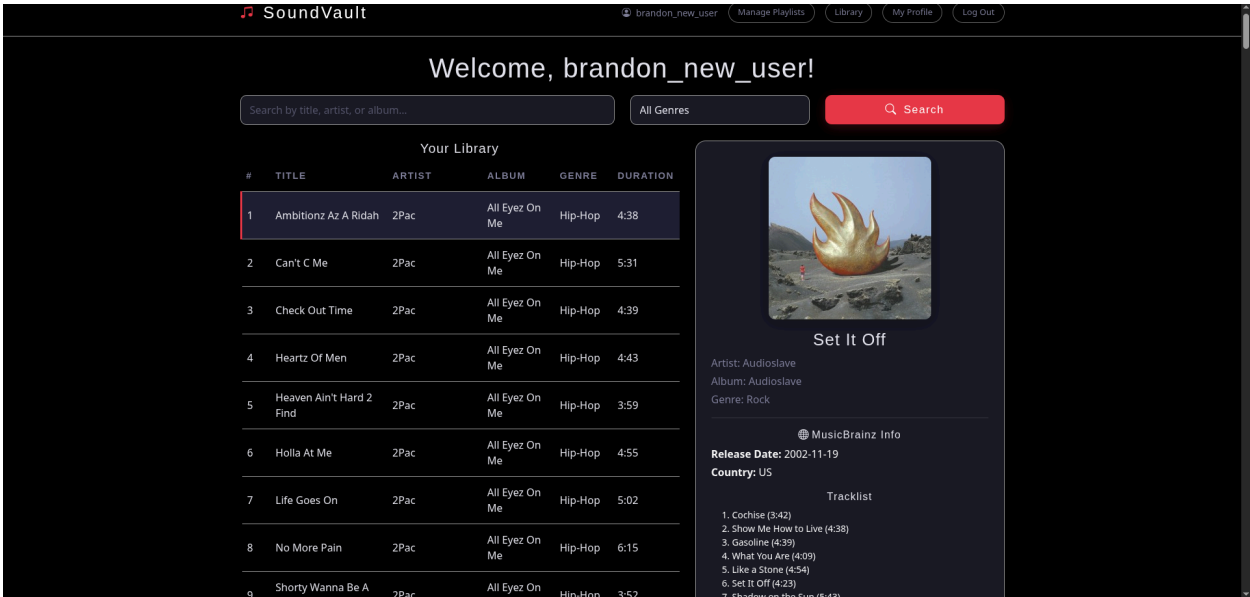


Home page:

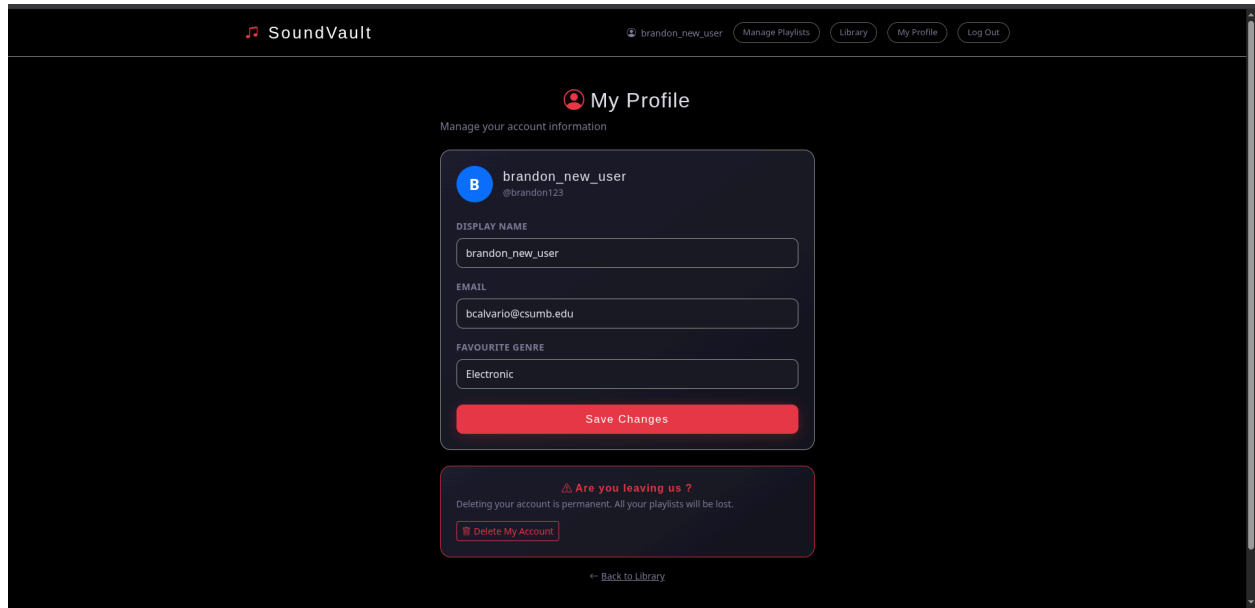
Brandon Calvario, Matthew Barrett, Maeva Akpaca, Dalia Cabrera Hurtado



Library page:



Profile page:



Team work:

Brandon worked on the backend, building the GET and POST routes and configuration for the API, server-side proxy streaming, Cloudflare R2 integration, session authentication, MusicBrainz integration, database implementation, and music file backend to be able to produce music output on the website securely.

Brandon worked on the Login and authentication setup (not signup) and made sure that routes were unable to be accessed by unauthorized users, as well as preventing users from being able to download content, preventing piracy.

Brandon managed the songs database by making sure the data of the songs is reflected accurately so the API would be able to reference the correct song to also match the song with the cloud database.

Brandon worked on scripts that allow importing songs from storage to cloud storage, and a separate script that enriches the data for each song in the database by matching the MusicBrainz information as mentioned above.

Brandon worked on building the library for the songs, displaying a limit of 200 songs in the list, as well as being able to play the songs through that route.

Brandon worked on the home/index page showing featured songs and individual songs in the index page to show a more engaging display of the database and API calling.

Maeve will handle the user profile pages and playlist management interface. This includes building the profile edit form (display name, email,...) with pre-filled fields that update the database via UPDATE SQL, the playlist creation/rename/delete, and the add/remove songs from playlist functionality. She will implement client-side JavaScript (Fetch API calls to the local Express routes) to interact with the backend, and handle session-based display of the logged-in user's data on the profile page. She will also contribute to the overall CSS styling for consistency across these pages.

Matthew will handle the playlist interface. This includes all the necessary files to allow users to interact with the song database in order to create their own playlists which will then also be added to a separate table. Playlists will have a name and, optionally, a description, which can be changed at any time, including after it is created. Users will be able to create as many playlists as they want, as well as adding as many songs to whichever playlist they choose. Users will also be able to remove songs from playlists as well as delete them altogether. This also includes all the relevant navigation to access the playlists as well as edit them.

Dalia Cabrera Hurtado: will be responsible for the music library browse pages, where users can explore music by artist, album, and genre with album art and metadata from the MusicBrainz API. I will also build the search feature so users can quickly find songs, artists, or albums. On top of that, I will create the home page and take charge of the overall CSS and site design to keep everything looking consistent across all pages.

Current datasets:

Songs Table:

The songs table was used to record the matching information from both the API and the cloud storage database.

Brandon Calvario, Matthew Barrett, Maeva Akpaca, Dalia Cabrera Hurtado

#	103 - song_id	A2 title	artist	A2 album	A2 genre	123 duration_sec	A2 cloud_key	A2 mb_release_id
1		Bring Em Back Alive	Audioslave	Audioslave	Rock	329	music/audioslave---bring-em-back-alive.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
2	2	Cochise	Audioslave	Audioslave	Rock	222	music/audioslave--cochise.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
3	3	Exploder	Audioslave	Audioslave	Rock	208	music/audioslave--exploder.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
4	4	Gasoline	Audioslave	Audioslave	Rock	279	music/audioslave--gasoline.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
5	5	Getaway Car	Audioslave	Audioslave	Rock	299	music/audioslave--getaway-car.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
6	6	Hypnotize	Audioslave	Audioslave	Rock	207	music/audioslave--hypnotize.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
7	7	I Am the Highway	Audioslave	Audioslave	Rock	335	music/audioslave--i-am-the-highway.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
8	8	Light My Way	Audioslave	Audioslave	Rock	303	music/audioslave--light-my-way.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
9	9	Like a Stone	Audioslave	Audioslave	Rock	294	music/audioslave--like-a-stone.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
10	10	Set It Off	Audioslave	Audioslave	Rock	263	music/audioslave--set-it-offexplicit.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
11	11	Shadow on the Sun	Audioslave	Audioslave	Rock	343	music/audioslave--shadow-on-the-sun.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
12	12	Show Me How to Live	Audioslave	Audioslave	Rock	278	music/audioslave--show-me-how-to-live.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
13	13	The Last Remaining Light	Audioslave	Audioslave	Rock	317	music/audioslave--the-last-remaining-light.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
14	14	What You Are	Audioslave	Audioslave	Rock	249	music/audioslave--what-you-are.flac	07fc4a1d-aef7-484f-a23b-10a6c8fc0b1d
15	15	(Don't Fear) The Reaper	Blue Oyster Cult	Agents Of Fortune	Rock	308	music/blue-yster-cult---dont-fear-the-reaper.m4a	5c881a7b-521c-44a2-9155-9ec07012b49e
16	16	After Dark	Blue Oyster Cult	Fire of Unknown Origin	Pop	268	music/blue-yster-cult---after-dark.m4a	93c38b19-f6d7-31ba-ad13-66c5a9938067
17	17	Before the Kiss, a Redcap	Blue Oyster Cult	Blue Oyster Cult	Rock	298	music/blue-yster-cult---before-the-kiss-a-redcap.m4a	29834783-c8f4-4058-b9b4-6b19ffc42560
18	18	Burnin' for the Kiss	Blue Oyster Cult	Fire of Unknown Origin	Pop	271	music/blue-yster-cult---burnin-for-you.m4a	93c38b19-f6d7-31ba-ad13-66c5a9938067
19	19	Celestial the Queen	Blue Oyster Cult	Spectres	Pop	206	music/blue-yster-cult---celestial-the-queen.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
20	20	Cities On Flame with Rock and Roll	Blue Oyster Cult	Blue Oyster Cult	Rock	243	music/blue-yster-cult---cities-on-flame-with-rock-and-roll	29834783-c8f4-4058-b9b4-6b19ffc42560
21	21	Death Valley Nights	Blue Oyster Cult	Spectres	Pop	248	music/blue-yster-cult---death-valley-nights.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
22	22	Debbie Denise	Blue Oyster Cult	Agents Of Fortune	Rock	253	music/blue-yster-cult---debbie-denise.m4a	5c881a7b-521c-44a2-9155-9ec07012b49e
23	23	Don't Turn Your Back	Blue Oyster Cult	Fire of Unknown Origin	Pop	248	music/blue-yster-cult---dont-turn-your-back.m4a	93c38b19-f6d7-31ba-ad13-66c5a9938067
24	24	E.T. (Extra Terrestrial Intelligence)	Blue Oyster Cult	Agents Of Fortune	Rock	223	music/blue-yster-cult---e.t.-extra-terrestrial-intelligence	5c881a7b-521c-44a2-9155-9ec07012b49e
25	25	Fire of Unknown Origin	Blue Oyster Cult	Fire of Unknown Origin	Pop	251	music/blue-yster-cult---fire-of-unknown-origin.m4a	93c38b19-f6d7-31ba-ad13-66c5a9938067
26	26	Fireworks	Blue Oyster Cult	Spectres	Pop	192	music/blue-yster-cult---fireworks.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
27	27	Godzilla	Blue Oyster Cult	Spectres	Pop	221	music/blue-yster-cult---godzilla.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
28	28	Goin' Through the Motions	Blue Oyster Cult	Spectres	Pop	192	music/blue-yster-cult---goin-through-the-motions.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
29	29	Golden Age of Leather	Blue Oyster Cult	Spectres	Pop	352	music/blue-yster-cult---golden-age-of-leather.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
30	30	Heavy Metal: The Black and Silver	Blue Oyster Cult	Fire of Unknown Origin	Pop	198	music/blue-yster-cult---heavy-metal-the-black-and-silver	93c38b19-f6d7-31ba-ad13-66c5a9938067
31	31	I Love the Night	Blue Oyster Cult	Spectres	Pop	264	music/blue-yster-cult---i-love-the-night.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
32	32	I'm On the Lamb but I Ain't No Sheep	Blue Oyster Cult	Blue Oyster Cult	Rock	190	music/blue-yster-cult---im-on-the-lamb-but-i-aint-no-sheep	29834783-c8f4-4058-b9b4-6b19ffc42560
33	33	Joan Crawford	Blue Oyster Cult	Fire of Unknown Origin	Pop	294	music/blue-yster-cult---joan-crawford.m4a	93c38b19-f6d7-31ba-ad13-66c5a9938067
34	34	Morning Film	Blue Oyster Cult	Agents Of Fortune	Rock	270	music/blue-yster-cult---morning-film.m4a	5c881a7b-521c-44a2-9155-9ec07012b49e
35	35	Nosferatu	Blue Oyster Cult	Spectres	Pop	324	music/blue-yster-cult---nosferatu.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
36	36	R. U. Ready 2 Rock	Blue Oyster Cult	Spectres	Pop	221	music/blue-yster-cult---r-u-ready-2-rock.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
37	37	Redeemed	Blue Oyster Cult	Blue Oyster Cult	Rock	231	music/blue-yster-cult---redeemed.m4a	29834783-c8f4-4058-b9b4-6b19ffc42560
38	38	Screams	Blue Oyster Cult	Blue Oyster Cult	Rock	190	music/blue-yster-cult---screams.m4a	29834783-c8f4-4058-b9b4-6b19ffc42560
39	39	Searchin' for Celine	Blue Oyster Cult	Spectres	Pop	216	music/blue-yster-cult---searchin-for-celine.m4a	956ab217-6a68-42fe-abea-73c4d93e71fd
40	40	She's as Beautiful as a Foot	Blue Oyster Cult	Blue Oyster Cult	Rock	176	music/blue-yster-cult---shes-as-beautiful-as-a-foot.m4a	29834783-c8f4-4058-b9b4-6b19ffc42560

Users Table:

[illegible]

Playlists table:

[illegible]

Playlist songs table (Table that holds data for playlist)

Timeline (insert what you did):

Friday, March 27th:

- Idea developed, rough draft of design brief submitted.
- Brandon started initial cloud storage-to-database testing

Tuesday, April 7th:

- Brandon shared github to the group.

Friday, April 16:

- Brandon implemented cloud backend, first song was tested and successfully played on the website
- Brandon pushed base/skeleton structure for login and songs has been built
-

Friday, April 21:

- Brandon implemented login functionality, but the user still requires creating account features and having info saved.
- Basic music playback functionality complete.

Sunday, April 26th:

- Brandon reorganized the program so that server.mjs holds all routes, this was before the lecture on modules, so I might revert it, if i have time..
-

Friday, May 1st:

- Brandon has implemented login functionality and proper redirection of cloud service and authentication.

Saturday, May 2nd:

- Matthew implemented the framework for what would be the playlist functionality

Sunday, May 3rd:

- Matthew added the rest of the functionality for playlists, including deleting, editing, adding songs, removing songs, and playing.
- Brandon edited the home page to separate the library from the main index, fixed some bugs, and made some bootstrap adjustments
- Maeva added profile functionality, allowing users to create and edit their profile

Monday, May 4th:

- Brandon made some adjustments to the main home page to give it more functionality
- Maeva added the ability for users to delete their own accounts, or for the admin to delete accounts

Tuesday, May 5th,

- Brandon implemented CSS to give the site a simple design.

API's Used:

https://musicbrainz.org/doc/MusicBrainz_API

A custom API, was not fully complete, but is the source of the songs via the cloud storage.